OWUS

maoyab/OWUS

english

gemini-2.5-flash

7bb8fe8

Dec 9, 2025

Chapters

Show Raw Markdown

Chapter 1: Ecohydrological Data Preparation

Welcome to the OWUS tutorial! In this chapter, we'll start our journey by learning how to gather and prepare all the necessary environmental data for our models. Think of this process like a specialized chef meticulously getting all the ingredients ready for a complex, gourmet recipe. Our "recipe" in the OWUS project is understanding how plants manage water, and the "ingredients" are various types of ecohydrological data.

Imagine you're a scientist wanting to study how different forests, grasslands, and c ecosystems across the globe cope with changes in water availability. To do this, yo on rainfall, how much water is in the soil, how much plants are growing, and even wh of soil they're in. This information comes from many different sources, in various formats, and often

has gaps or inconsistencies.

The problem: Raw environmental data is messy! It comes from different sensors, satellites, and databases, each with its own quirks. Before we can use it for scientific modeling, we need to:

1. **Collect**: Gather data from many diverse places.
2. **Clean**: Remove errors, fill gaps, and ensure data quality.
3. **Standardize & Prepare**: Convert units, choose specific time periods (like a plant's "growing season"), and calculate new useful measurements.

This chapter will guide you through how the `OWUS` project handles this "Ecohydrological Data Preparation" to turn raw, diverse environmental data into a neat, standardized format, ready for the next steps in our analysis.

How OWUS Prepares Data: A High-Level View

In the `OWUS` project, we typically process data for many research sites. Each site is like a unique "dish" that needs its own set of ingredients. For this tutorial, let's focus on preparing data for a single site, for example, the "AT-Neu" site, which is a grassland in Austria.

The main function that orchestrates this preparation is `pool_plot_data`, which lives in the `select_site_records.py` file. It's designed to process one site at a time, making sure all data is collected, cleaned, and parameters are calculated.

Let's see how you would trigger this process for a single site:

```
import pandas as pd
from select_site_records import pool_plot_data

# Load the list of sites we are interested in
sites_params = pd.read_csv('sel_sites.csv')

# Choose our example site
example_site_id = 'AT-Neu'

# Run the data preparation for this site
print(f"Preparing data for site: {example_site_id}")
pool_plot_data(example_site_id)
print(f"Data preparation complete for site: {example_site_id}")

```

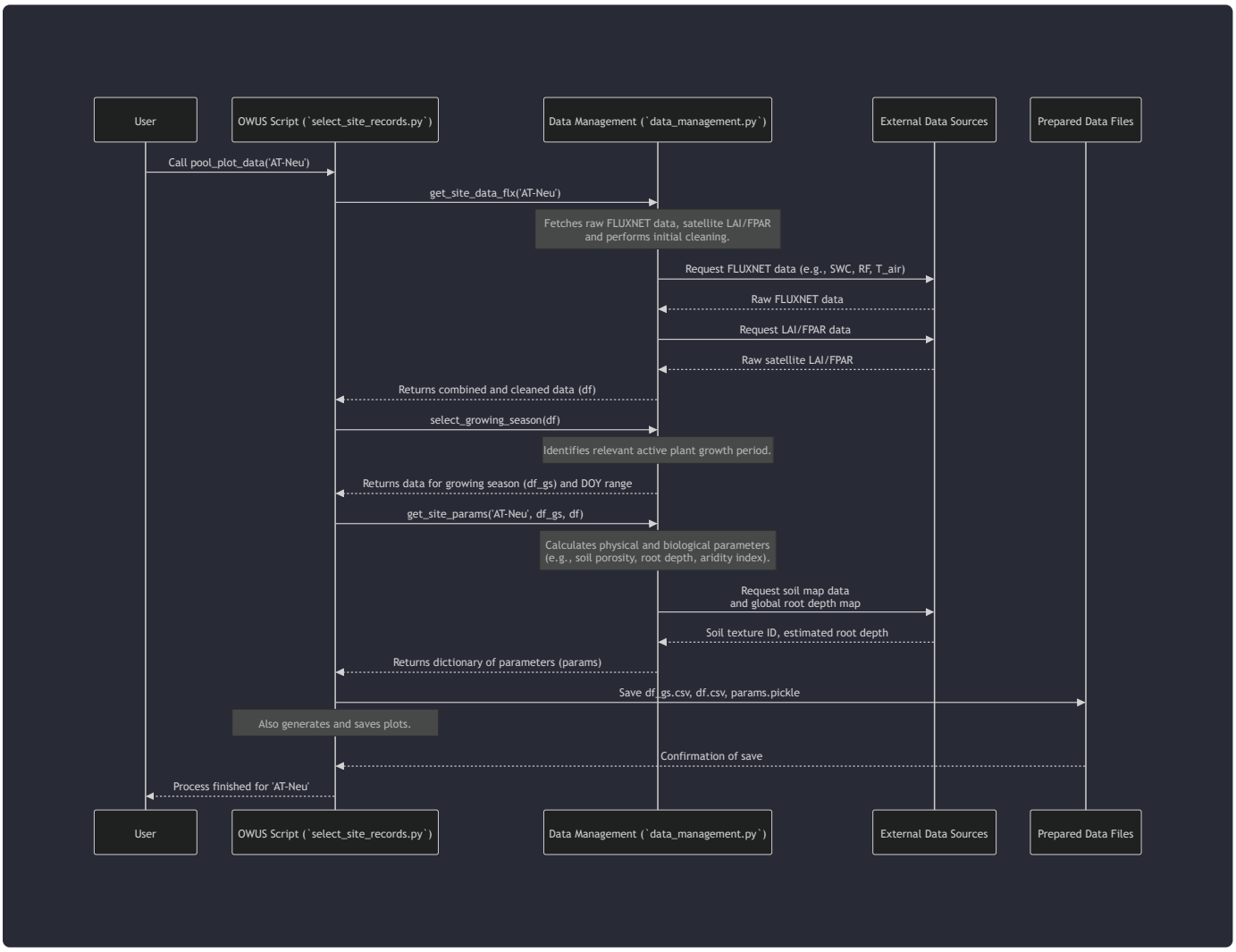
After running this, the `OWUS` system will:

1. Fetch raw data (like rainfall, temperature, soil moisture) for 'AT-Neu'.
2. Identify the main "growing season" for plants at this site.
3. Calculate various site-specific parameters (e.g., soil properties, root depth).
4. Save the prepared data and parameters into files in a designated `DATA/WUS/WUS_input_data` folder. It will also generate a plot summarizing the prepared data in `DATA/OWUS/plots/data_growing_seasons`.

This single function call wraps up a lot of behind-the-scenes work. Let's peek into how it does it!

Diving Deeper: The Preparation Steps

The data preparation process involves several key steps. Here's a simplified sequence of actions when `pool_plot_data` is called for a site:



Let's break down the core components mentioned in the diagram and the code.

1. Site Information: The Blueprint (`sel_sites.csv`)

Before we collect any data, we need to know *where* our site is and some basic facts about it. This information is stored in a simple CSV file called `sel_sites.csv` . It acts like an index for all our research sites.

Here's a small snippet from `sel_sites.csv` :

```
siteID,lat,lon,climate,pft_0,pft,IGBP,soil_tex_id,Soil_TEX,swc_i,Zm
AT-Neu,47.11667,11.3175,Te,H,G-C3,GRA,3,SANDY LOAM,2,-0.1
BR-Sa3,-3.01803,-54.97144,Tr,BL,BF-tr,EBF,9,CLAY LOAM,1,-0.15
CA-Oas,53.62889,-106.19779,Bo,BL,BF-te,DBF,3,SANDY LOAM,1,-0.1

```

Each row describes a site. For 'AT-Neu', we see its latitude, longitude, climate type (`Te` for Temperate), plant functional type (`G-C3` for C3 Grassland), soil texture ID (`3` for Sandy Loam), and the index of the soil water content sensor to use (`swc_i`). This file is the starting point for `get_site_params` to gather site-specific details.

2. Fetching & Cleaning Raw Data (`get_site_data_flx`)

The `get_site_data_flx` function (from `data_management.py`) is responsible for bringing in the raw measurements. It's like collecting all the fresh ingredients from various suppliers.

- **FLUXNET Data**: This is a global network of stations that measure exchanges of carbon dioxide, water vapor, and energy between ecosystems and the atmosphere. `get_site_data_flx` reads these measurements (like `SWC` for Soil Water Content, `RF` for Rainfall, `T_air` for Air Temperature) directly from the FLUXNET daily data files.
- **Satellite Observations**: We also integrate data from satellites, such as LAI (Leaf Area Index) and FPAR (Fraction of Absorbed Photosynthetically Active Radiation), which tell us about plant growth and canopy density.
- **Data Cleaning & Conversion**: The function cleans this raw data. For example, it filters out low-quality measurements using quality control (QC) flags provided by FLUXNET, converts units (e.g., kPa to Pa for VPD), and calculates basic derived variables like potential evaporation (`ETo_m`).

Here's a simplified look at how `get_site_data_flx` processes `SWC` and `LAI` :

```python
# From data_management.py (simplified)
def get_site_data_flx(site, swc_i=None, x_years=True):
    # ... (code to find and read FLUXNET file) ...
```

```
    # Process Soil Water Content (SWC)
    data['SWC_F_MDS'] = data['SWC_F_MDS_%d' % swc_i] # Select specific SWC sensor
    data['SWC_F_MDS_QC'] = data['SWC_F_MDS_%d_QC' % swc_i]
    data.loc[data['SWC_F_MDS_QC'] < 0.5, 'SWC_F_MDS'] = np.nan # Remove low-quali
    data['SWC'] = data['SWC_F_MDS'] / 100. # Convert percentage to fraction

    # ... (process other variables like VPD, P_air, T_air, RF, GPP, ETo) ...

    # Insert Leaf Area Index (LAI) from satellite data
    # The 'insert_LAI' function handles loading and smoothing LAI data
    data = insert_LAI(data, site_name, df_p, df_lai)

    data = data.dropna() # Remove any remaining rows with missing values
    return data

```

This function is crucial because it ensures we're working with reliable measurements in consistent units.

3. Identifying the Growing Season (`select_growing_season`)

Plants don't actively grow all year round. In many climates, they have a specific "growing season." Our models need to focus on this active period to accurately simulate plant water use. The `select_growing_season` function (also in `data_management.py`) automatically figures out these crucial months or days of the year.

It does this by looking at indicators of plant activity, primarily the `LAI` (Leaf Area Index) or `GPP` (Gross Primary Production, a measure of photosynthesis) and ensuring the `T_air` (Air Temperature) is above a certain threshold.

```
# From data_management.py (simplified)
def select_growing_season(data, gpp_th=[95, 0.10], t_th=2, lai_th=None):
    data['DOY'] = data.index.dayofyear
    data['M'] = data.index.month

    # Calculate monthly averages for LAI, GPP, and T_air
    data_m = data[['GPP', 'LAI', 'T_air', 'M']].groupby('M').mean().dropna()

    # Determine growing months based on LAI or GPP and temperature thresholds
    if lai_th is None:
```

```
        # Use GPP threshold (e.g., 10% of the 95th percentile GPP)
        th_m = np.percentile(data_m['GPP'].dropna(), gpp_th[0]) * gpp_th[1]
        data_m = data_m[(data_m['GPP'] >= th_m) & (data_m['T_air'] >= t_th)]
        months_gs = list(data_m.index)
    else:
        # Use LAI threshold (e.g., 10% of the 90th percentile LAI)
        th_m = np.percentile(data_m['LAI'].dropna(), lai_th[0]) * lai_th[1]
        data_m = data_m[(data_m['LAI'] >= th_m) & (data_m['T_air'] >= t_th)]
        months_gs = list(data_m.index)

    # Filter the original data to only include the identified growing season month
    selected_days = [i for i in data.index if i.month in months_gs]
    data_gs = data.loc[selected_days]

    # ... (additional filtering for years with sufficient data) ...

    return data_gs, doy_gs # Returns growing season data and DOY range

```

By selecting only the growing season, we ensure our models focus on the periods when plants are actively using water, making the simulations more relevant and accurate.

4. Calculating Site Parameters (`get_site_params`)

The `get_site_params` function (also in `data_management.py`) calculates or fetches all the crucial numbers (parameters) that describe the soil, plants, and climate at our specific site. These parameters are essential inputs for the Stochastic Soil Water Balance Model (SSWM) in the next chapter.

It gathers:

- **Climate characteristics**: Mean rainfall, aridity index, vapor pressure deficit.
- **Soil physical characteristics**: Porosity, field capacity, wilting point (how much water the soil can hold and how much is available to plants). These are derived using the `soil_tex_id` from `sel_sites.csv` and external soil property databases.
- **Root characteristics**: Rooting depth, obtained from global maps or plant functional type (PFT) averages.
- **Plant characteristics**: Canopy height, maximum xylem conductivity, stomatal conductance (how much water plants can transport and release). These often come from general values associated with the plant functional type (`pft` column in `sel_sites.csv`).

- **Surface characteristics**: LAI and FPAR values (from satellite data), and fractions of different vegetation types (e.g., trees, herbaceous plants).

Here's a simplified glimpse into `get_site_params`, focusing on soil properties and porosity:

```python
# From data_management.py (simplified)
def get_site_params(site, data, data_0, swc_i=None):
    # ... (code to load lat, lon, soil_tex_id, pft from sites_params) ...

    params = {
        'siteID': site,
        'lat': lat,
        'lon': lon,
        'soil_tex_id': soil_tex_id,
        'Zm': -zm, # Measurement depth of SWC sensor
        # ... (other basic site info) ...
    }

    # Retrieve soil physical characteristics from a database based on soil_tex_id
    # BB: Clapp and Hornberger exponent; SATPSI: Saturated matric potential;
    # SATDK: Saturated hydraulic conductivity; MAXSMC: Saturated soil moisture cor
    # DRYSMC: Dry soil moisture content
    BB, SATPSI, SATDK, MAXSMC, DRYSMC = get_gldas_soil_data(soil_tex_id)

    params['b'] = BB # Soil texture parameter (Brooks-Corey exponent)
    params['Ps0'] = SATPSI # Soil texture parameter (Saturated soil matrix potenti
    params['Ks'] = SATDK * params['Td'] # Saturated hydraulic conductivity

    # Estimate porosity (n) from the maximum observed soil water content
    swcmax = np.ceil(np.nanmax(data_0['SWC']) * 100) / 100.
    params['n'] = swcmax # Total porosity of the soil

    # Calculate field capacity (s_fc) and permanent wilting point (s_h)
    # These represent the upper and lower limits of plant-available water
    params['s_h'] = pot_to_s_BC(-10, params['b'], params['Ps0']) # Wilting point
    params['s_fc_ref'] = pot_to_s_BC(-0.03, params['b'], params['Ps0']) # Referenc
    # Also calculate an observed field capacity based on soil moisture peaks after
    data_0['ds/dt'] = data_0['S'].diff()
    sm_peaks = [smi for dsi, smi in zip(data_0['ds/dt'].values, data_0['S'].values
    s_fc_r = np.percentile(sm_peaks, 95)
    params['s_fc_rec'] = s_fc_r
    params['s_fc'] = np.max([params['s_fc_ref'], params['s_fc_rec']]) # Final fie
```

```
38
39      # ... (code to estimate root depth, plant hydraulic parameters, etc.) ...
40
41      return params
42
```

This comprehensive set of parameters ensures that the Stochastic Soil Water Balance Model (SSWM) is tailored to the unique conditions of each site.

Conclusion

In this chapter, we've learned that "Ecohydrological Data Preparation" is the crucial first step in any environmental modeling project. It's like having a dedicated team of experts (our `OWUS` code) collecting, cleaning, and organizing all the raw ingredients before the actual cooking (modeling) begins. We saw how `OWUS` takes messy, diverse data from sources like FLUXNET and satellites, processes it to focus on relevant periods like the growing season, and derives important site-specific parameters.

This prepared data and these carefully calculated parameters are now ready to be fed into our core hydrological model.

Are you ready to see how this data is used to simulate the movement of water in the soil? Then let's move on to the next chapter!

Next Chapter: Stochastic Soil Water Balance Model (SSWM)

Generated by AI Codebase Knowledge Builder. **References**: [1], [2], [3]